

Assignment-19.1

Name: N.Srishanth

Hall.No: 2303A51907

Batch – 12

Task 1 – Python to Java Conversion

Task:

Translate a simple Python class into Java using AI assistance.

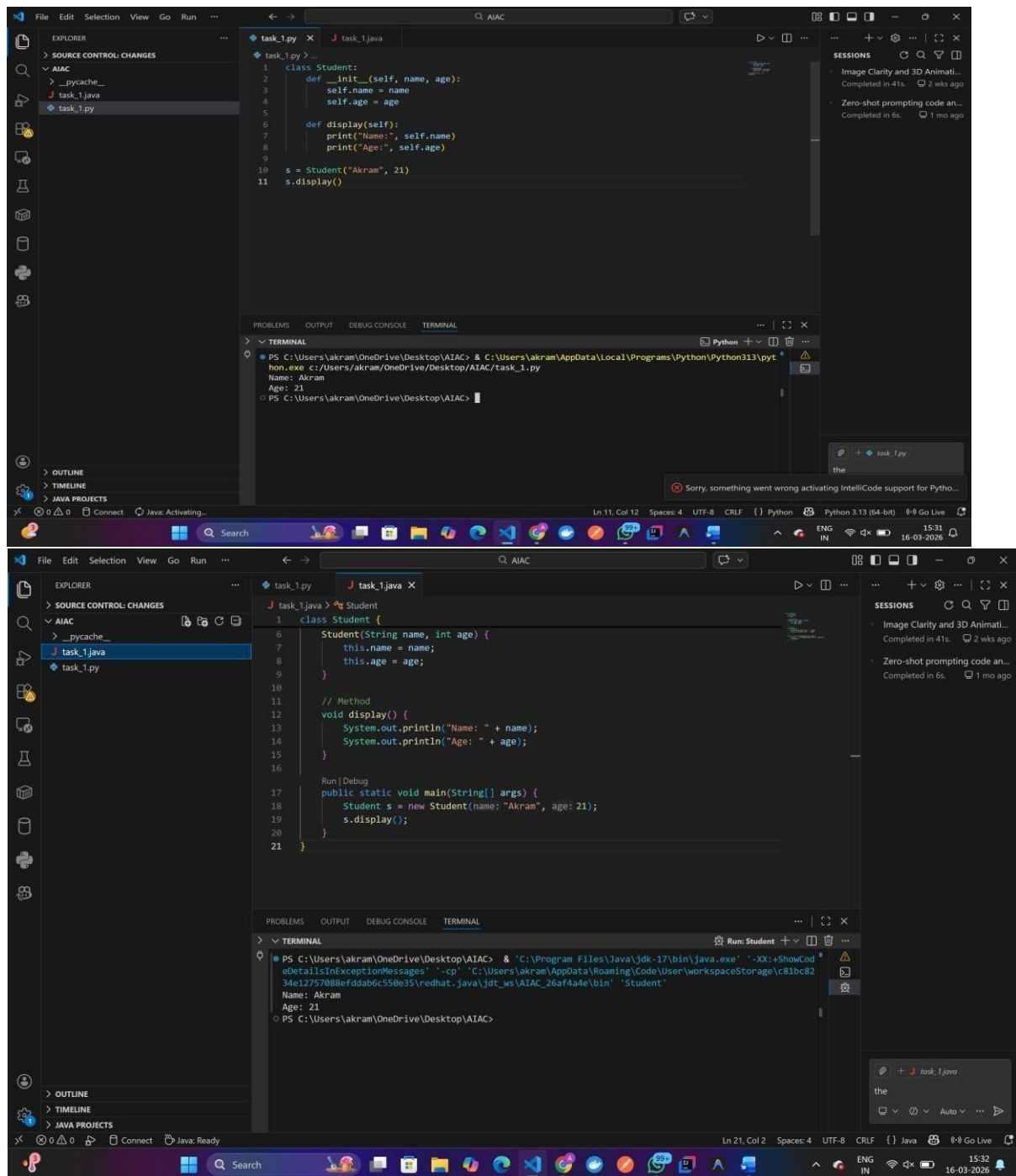
Instructions:

- Prompt AI to convert a Python class with attributes and methods into equivalent Java code.
- Verify that constructors, method signatures, and data types are properly handled.
- Run both versions (Python & Java) to compare expected outputs.

Expected Output:

- Equivalent working Java class translated from Python.

CODE:



Task 2 – C++ to Python Function Conversion

Task:

Convert a C++ function for factorial calculation into Python.

Instructions:

- Ask AI to translate a recursive C++ factorial function into Python.

- Validate correctness by testing with sample inputs.
- Ensure Python version follows Pythonic conventions.

Expected Output:

- A correct Python implementation equivalent to the original C++ factorial function.

CODE:

The screenshot shows the Visual Studio Code editor with a Python file named `task2.py` open. The code defines a recursive factorial function and prints the result for 5. The terminal shows the execution output.

```

1 def factorial(n):
2     if n == 0:
3         return 1
4     else:
5         return n * factorial(n-1)
6
7 print(factorial(5))

```

```

PS C:\Users\akram\OneDrive\Desktop\AIAC> & C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:\Users\akram\OneDrive\Desktop\AIAC\task_1.py
Name: Akram
Age: 21
PS C:\Users\akram\OneDrive\Desktop\AIAC> & C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:\Users\akram\OneDrive\Desktop\AIAC\task2.py
120
PS C:\Users\akram\OneDrive\Desktop\AIAC>

```

The screenshot shows the Visual Studio Code editor with a C++ file named `task_2.cpp` open. The code defines a recursive factorial function and prints the result for 5. The terminal shows the execution output.

```

1 #include <iostream>
2 using namespace std;
3
4 int factorial(int n) {
5     if(n == 0)
6         return 1;
7     else
8         return n * factorial(n-1);
9 }
10
11 int main() {
12     cout << factorial(5);
13     return 0;
14 }

```

```

PS C:\Users\akram\OneDrive\Desktop\AIAC> & C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:\Users\akram\OneDrive\Desktop\AIAC\task_1.py
Name: Akram
Age: 21
PS C:\Users\akram\OneDrive\Desktop\AIAC> & C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:\Users\akram\OneDrive\Desktop\AIAC\task2.py
120
PS C:\Users\akram\OneDrive\Desktop\AIAC>

```

Task 3 – SQL to Pandas Query

Task:

Translate a SQL query into a Pandas DataFrame operation in Python.

Instructions:

- Provide AI with a SQL query (e.g., `SELECT name, salary FROM employees WHERE salary > 50000`).
- Ask AI to generate equivalent Pandas code.
- Test the generated code on a sample DataFrame.

Expected Output:

- Equivalent Pandas query that retrieves the correct subset of data.

CODE:

The screenshot shows the VS Code editor with a Python file named `task_3.py` open. The code imports `pandas` and creates a DataFrame with employee data. The terminal output shows the execution of the script, which prints the names and salaries of employees earning more than 50,000.

```
task_3.py > ...
1 import pandas as pd
2
3 data = {
4     'name': ['John', 'Alice', 'Bob', 'Emma'],
5     'salary': [45000, 60000, 75000, 40000]
6 }
7
8 df = pd.DataFrame(data)
9
10 result = df[df['salary'] > 50000][['name', 'salary']]
11
12 print(result)
```

```
PS C:\Users\akram\OneDrive\Desktop\AIAC> & C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:\Users\akram\OneDrive\Desktop\AIAC\task_3.py
Age: 21
120
PS C:\Users\akram\OneDrive\Desktop\AIAC> & C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:\Users\akram\OneDrive\Desktop\AIAC\task2.py
120
PS C:\Users\akram\OneDrive\Desktop\AIAC> & C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:\Users\akram\OneDrive\Desktop\AIAC\task_3.py
   name  salary
1  Alice  60000
2    Bob   75000
PS C:\Users\akram\OneDrive\Desktop\AIAC>
```

The screenshot shows the VS Code editor with a SQL file named `task_3.sql` open. The code contains a SQL query to select employee names and salaries. The terminal output shows the execution of the script, which prints the results of the query.

```
task_3.sql
1 SELECT name, salary
2 FROM employees
3 WHERE salary > 50000;
```

```
PS C:\Users\akram\OneDrive\Desktop\AIAC> & C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:\Users\akram\OneDrive\Desktop\AIAC\task_3.py
Age: 21
PS C:\Users\akram\OneDrive\Desktop\AIAC> & C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:\Users\akram\OneDrive\Desktop\AIAC\task2.py
120
PS C:\Users\akram\OneDrive\Desktop\AIAC> & C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:\Users\akram\OneDrive\Desktop\AIAC\task_3.py
   name  salary
1  Alice  60000
2    Bob   75000
PS C:\Users\akram\OneDrive\Desktop\AIAC>
```

Task 4 – Real-Time Application: Multi-Language Snippet

Converter Scenario:

Students will choose a real-time use case where they need code in multiple languages. Example:

- Converting a Python web scraping snippet into JavaScript for browser automation.

- Translating a Java method into C# for enterprise applications.

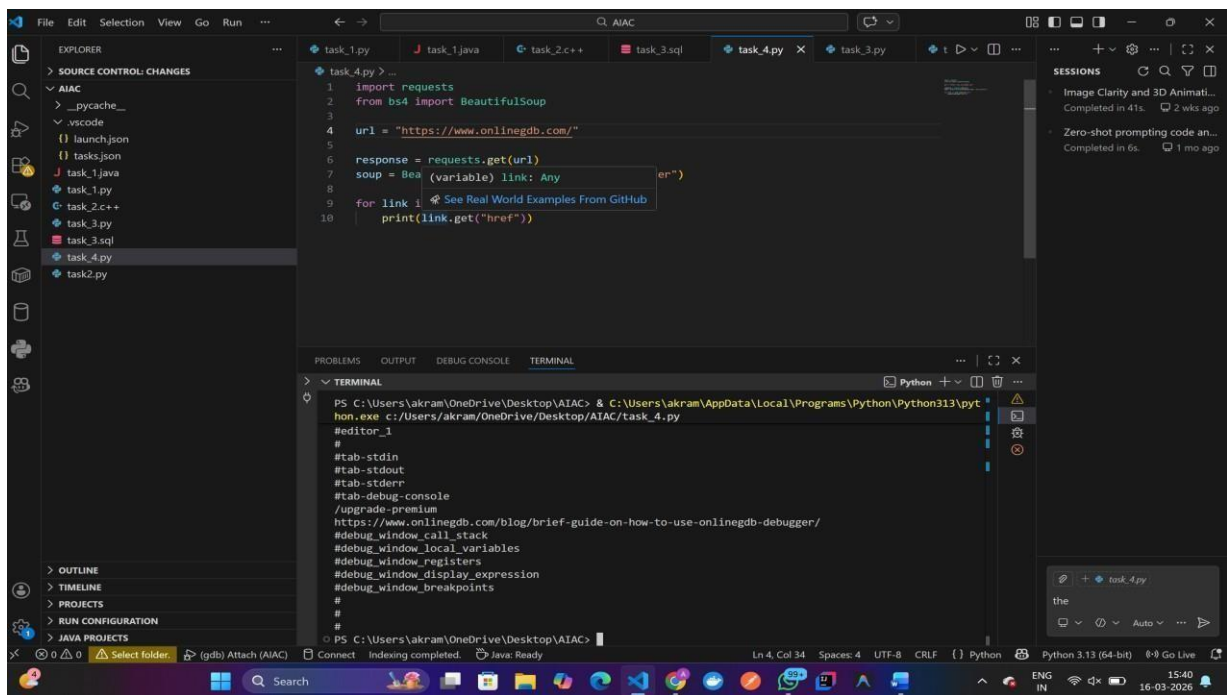
Instructions:

- Prompt AI to translate code between two chosen languages.
- Test both implementations to ensure they produce the same results.
- Document challenges faced during translation (e.g., syntax, libraries).

Expected Output:

- Working equivalent code snippets in at least two different programming languages

CODE:



The screenshot shows the Visual Studio Code interface. The Explorer pane on the left shows a project named 'AIAC' with files like 'task_1.py', 'task_2.c++', 'task_3.py', 'task_3.sql', 'task_4.py', and 'task2.py'. The main editor window displays the content of 'task_4.py', which is a Python script using the 'requests' and 'BeautifulSoup' libraries to fetch data from a URL and print the href of the first link. The script is as follows:

```
1 import requests
2 from bs4 import BeautifulSoup
3
4 url = "https://www.onlinegdb.com/"
5
6 response = requests.get(url)
7 soup = BeautifulSoup(response.text, 'html.parser')
8 for link in soup.find_all('a'):
9     print(link.get('href'))
10
```

The TERMINAL pane at the bottom shows the command prompt output after running the script. It displays the file path, the command to run the script, and the output of the script, which is the href of the first link found on the page: 'https://www.onlinegdb.com/blog/brief-guide-on-how-to-use-onlinegdb-debugger/'.

```
PS C:\Users\akram\OneDrive\Desktop\AIAC> & C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:\Users\akram\OneDrive\Desktop\AIAC\task_4.py
#editor_1
#
#tab-stdin
#tab-stdout
#tab-stderr
#tab-debug-console
/upgrade-premium
https://www.onlinegdb.com/blog/brief-guide-on-how-to-use-onlinegdb-debugger/
#debug_window_call_stack
#debug_window_local_variables
#debug_window_registers
#debug_window_display_expression
#debug_window_breakpoints
#
#
```

